Higher National School of Advanced Technologies (ENSTA)

SS/IA: First Semester - 2026

Module: WAMD

TD -TP 3: Build an E-Commerce Backend: REST API with Node.js, Express.js, MongoDB, JWT, OAuth2, and rate limiting

1. Lab Objectives

At the end of this lab, students should be able to:

- Create a basic backend application using **Node.js**.
- Create REST API endpoints using **Express.js**.
- Understand the basic principles of a REST API.
- Connect a Node.js application to **MongoDB**.
- Create and manipulate basic data stored in MongoDB.
- Implement basic **CRUD operations** through an API.
- Understand the purpose of **JWT authentication**.
- Generate a JWT token after successful authentication.
- Protect API endpoints using JWT authentication and OAuth2.
- Test REST API endpoints using a suitable API testing tool.
- Organize a simple backend project into separate files.
- Handle basic errors and HTTP status codes.

2. Tools and Technologies

Tool / Technology	Purpose
Visual Studio Code	Development environment
Node.js	Backend runtime environment
Express.js	Web framework for the REST API
MongoDB	NoSQL database
MongoDB Atlas / MongoDB Community Server	MongoDB environment
Mongoose	MongoDB interaction from Node.js
JavaScript	Backend programming
Postman / Thunder Client	API testing
Git / GitHub	Version control and project submission

3. Lab Statement

Problem

In TD–TP 2, you developed a small e-commerce frontend using React and Redux. The application displays products and allows users to manage a shopping cart. However, the product information is currently stored locally in the frontend. There is no backend server, database or user authentication. Your task in this lab is to develop the backend for the same e commerce application. The backend must provide a REST API using Node.js, Express.js and MongoDB. It will allow the frontend to retrieve product information from the database and provide basic user registration and login functionality. The backend must also use JWT

authentication and OAuth2, to protect selected API endpoints. The resulting architecture will be:

Proposed by: Dr. S. SENHADJI
Higher National School of Advanced Technologies (ENSTA)

SS/IA: First Semester - 2026

Module: WAMD



The React frontend does not connect directly to MongoDB. It communicates with the backend through HTTP requests.
Important: This lab focuses on the backend. Students are not required to rebuild their React interface.

4. Functional Requirements

A. Product Management

The API must manage the products displayed in the e-commerce application.
Each product should contain appropriate information, such as:

- Product name
- Description
- Price
- Category
- Image URL
- Stock quantity

The API must allow clients to:

- Retrieve all products.
- Retrieve a product by its ID.
- Create a product.
- Update a product.
- Delete a product.

Products must be stored in MongoDB rather than maintained only in a JavaScript array. **B. User Registration and Login**

The API must allow users to create accounts and log in.

Registration

- Receive user information.
- Validate the required fields.
- Hash the password before storing it.
- Store the user in MongoDB.
- Prevent duplicate accounts using the same email address.

Login

- Receive the user's email and password.
- Verify the credentials.
- Generate a JWT after successful authentication.
- Return the token to the client.

Passwords must never be stored as plain text.

C. JWT Authentication

The backend must use JWT authentication to protect selected endpoints.

When a client sends a protected request, it must include the token in the HTTP request header. For this introductory lab, students may protect product creation, modification and deletion endpoints. Product retrieval can remain public.

Role-based authorization, such as separate administrator and customer roles, is not required.

Proposed by: Dr. S. SENHADJI
Higher National School of Advanced Technologies (ENSTA)

5. REST API Requirements

The API should follow REST principles and use appropriate HTTP methods and status codes. The following endpoint structure is recommended.

Authentication endpoints:

Method	Endpoint	Purpose
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Authenticate a user and return a JWT

Product endpoints:

Method	Endpoint	Purpose	Access
GET	/api/products	Retrieve all products	Public
GET	/api/products/:id	Retrieve one product	Public
POST	/api/products	Create a product	JWT required
PUT	/api/products/:id	Update a product	JWT required
DELETE	/api/products/:id	Delete a product	JWT

			required

The access restrictions above are a simplified teaching setup. In a production e-commerce application, product management would normally require additional authorization, such as administrator permissions.

You should return appropriate status codes, including:

- 200 OK — successful retrieval or update.
- 201 Created — successful creation.
- 400 Bad Request — invalid input.
- 401 Unauthorized — missing or invalid authentication.
- 404 Not Found — resource not found.
- 409 Conflict — duplicate email, where appropriate.
- 500 Internal Server Error — unexpected server error.

6. MongoDB Requirements

The application must use MongoDB or MongoDB Atlas to store product and user information. You should:

- Create a database for the e-commerce application.
- Connect the backend to MongoDB using Mongoose.
- Define a Product model.
- Define a User model.
- Insert and retrieve product documents.
- Update and delete product documents.
- Store user information securely, including hashed passwords.

A simplified data structure could be:

Product

Represents an item available in the store.

name · description · price · category · imageUrl · stock

User

Represents a registered customer.

name · email · passwordHash

you can populate the database with a small collection of sample products so the API can be tested immediately.

7. JWT Authentication Workflow

The authentication process should follow this general principle:

User submits login credentials

Express verifies credentials

Generate and return JWT

Client sends JWT with protected requests

Middleware verifies the token

Valid

Continue request

Invalid

Return 401

You should implement middleware that extracts the token from the Authorization header, verifies it and permits or rejects the request accordingly.
The token should be sent using the following format:
Authorization: Bearer <JWT_TOKEN>
The JWT secret must be stored in an environment variable, not hardcoded in the application.

8. OAuth 2.0 Authentication Testing with Postman (Thunder Client)

The OAuth 2.0 authentication process should follow this general principle:
Configure an OAuth 2.0 provider and obtain the required credentials
Open Postman or Thunder Client in VS Code
Select OAuth 2.0 as the authorization type
Configure the authorization URL, token URL, client ID, client secret (if required), and redirect URI Request authorization and authenticate with the provider
Obtain the access token
Send a request to a protected API endpoint using the token
Valid token → Continue request
Invalid or missing token → Return 401 Unauthorized when authentication is required You should configure the appropriate OAuth 2.0 flow supported by the provider. For a user sign-in scenario, use OpenID Connect with Authorization Code flow and PKCE where supported. Configure the redirect URI exactly as registered with the provider.
The access token should be sent using the following format:
Authorization: Bearer <ACCESS_TOKEN>

9. Rate Limiting for API Security

Rate limiting restricts the number of requests a client can send to the API within a specified period. It helps reduce excessive requests and brute-force login attempts.
You should:

- Install and configure the express-rate-limit package.
- Apply a general rate limit to API endpoints.
- Configure a stricter rate limit for the login endpoint.
- Return an appropriate error response when the request limit is exceeded.
- Test rate limiting using Postman or Thunder Client.

For this lab, use the following configuration as a starting point:

- General API: Maximum 100 requests per 15 minutes.
- Login endpoint: Maximum 5 requests per 15 minutes per client IP.

When the limit is exceeded, the API should return 429 Too Many Requests.
You should test the implementation by sending repeated requests to the product and login endpoints, verifying the responses, and providing screenshots demonstrating the rate-limiting behavior.

10. API Testing

You must test API using Postman or Thunder Client.
The tests should include:
Authentication

- Register a new user.
- Attempt to register with an existing email.
- Log in with valid credentials.

- Attempt to log in with invalid credentials.
- Receive a JWT after successful login.

Product management

- Retrieve the product list.
- Retrieve a single product.
- Create a product with a valid token.
- Update a product with a valid token.
- Delete a product with a valid token.

Security and error handling

- Attempt a protected request without a token.
- Attempt a protected request with an invalid token.
- Test an invalid product ID.
- Test missing or invalid product information.

You should provide screenshots demonstrating the results.

11. Preparation for React Integration

The API must be designed so that the React application from TD–TP 2 can consume it in the next stage.

You should ensure that:

- Product endpoints return JSON.
- Product fields are clearly defined and consistent.
- The product list can be retrieved using an HTTP GET request.
- The API accepts JSON when creating or updating products.
- The backend handles requests from the React development origin using appropriate CORS configuration.
- Errors are returned in a consistent, understandable format.

For this lab, Postman testing is sufficient. you do not need to connect the React frontend to the backend yet unless you want to make that an optional extension.

The shopping cart can remain managed by Redux in TD–TP 2. Persisting carts or implementing real orders is outside the scope of TD–TP 3.

12. Deliverables

You should submit their work through the course Classroom.

The submission must contain:

1. GitHub repository link containing the complete backend project.
2. Postman or Thunder Client screenshots demonstrating the required API operations and authentication tests.
3. README.md containing:
    - Student name.
    - Application description.
    - Technologies used.
    - Database models.
    - API endpoints and access requirements.
    - Description of the JWT authentication mechanism.
    - Instructions for installing dependencies and running the project.
    - Instructions for configuring MongoDB and environment variables.

Proposed by: Dr. S. SENHADJI
Higher National School of Advanced Technologies (ENSTA)

Important security requirement

You must not commit real database credentials, passwords or JWT secrets to GitHub.The .env file must be excluded through .gitignore. you should submit an .env.example file containing placeholders,

for example:
PORT=3000
MONGODB_URI=your_mongodb_connection_string
JWT_SECRET=replace_with_a_local_secret